

How Do We Deal With Missing Modality In Multi-Omics Data

When one of the Modalities associated with patient is missing

Gautam Anand | Yash Prakash Gupta | Gunjan Gupta

Table Of Contents

Part I: Context & Problem

- Introduction
- Outline
- Previous Work and Baselines
- Motivation

Part II: Solution & Results

- Proposed Architecture
- Experimental Results
- Conclusion
- Future Work
- References

Outline

- 1. Problem:** Missing modalities in multi-omics data can lead to loss of patients or unreliable information.
- 2. Reconstruction:** Reconstruct missing DNA from available miRNA using a trained predictor.
- 3. Confidence:** Assign a confidence score to distinguish observed DNA from reconstructed DNA.
- 4. Fusion & Classification:** Integrate DNA, mRNA and miRNA using confidence-aware PMMHA/MAGNET fusion for cancer classification.
- 5. Experiments:** Evaluate the integrated framework on BRCA, BLCA and OV under Control, Stress A and Stress B conditions.
- 6. Results & Analysis:** Compare Accuracy and Macro-F1 with Original MAGNET and analyze robustness to confidence and noise.
- 7. Conclusion & Future Work:** Discuss limitations and directions for patient-specific confidence, unresolved modalities, and broader reconstruction.

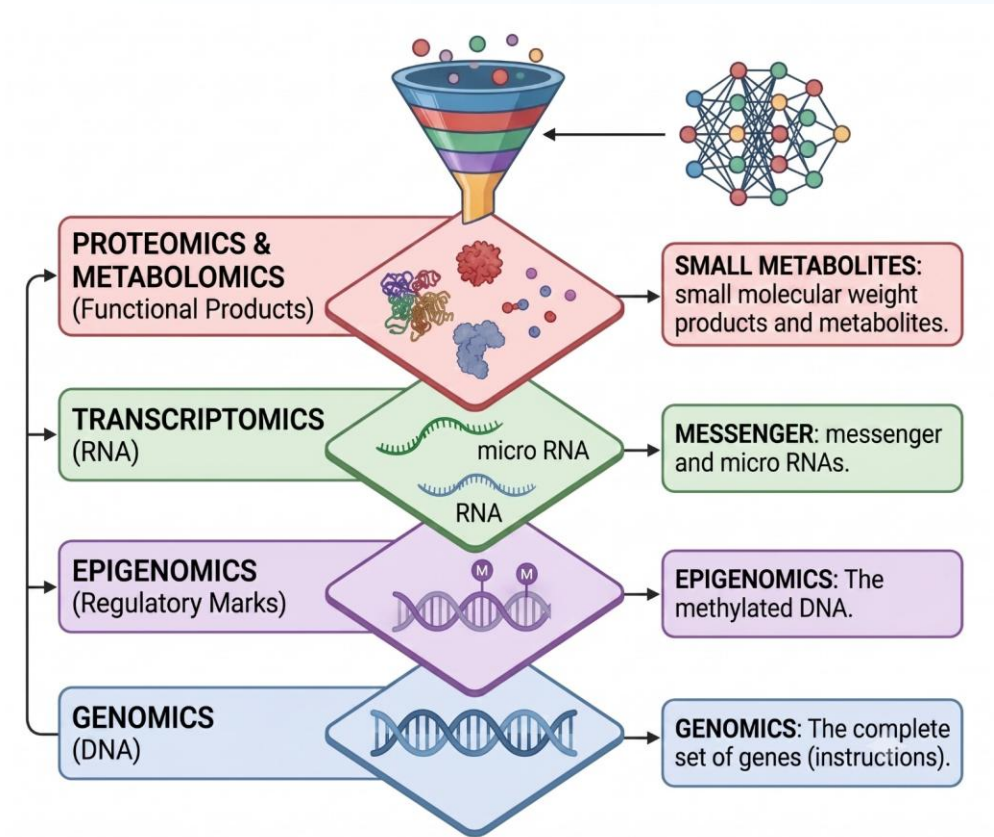
Intro & Arch. Bottleneck

What are Modalities?

Modalities are different types or sources of medical information used to understand a patient's health or disease – like X-ray reports, DNA expression, biopsy images, other patient records.

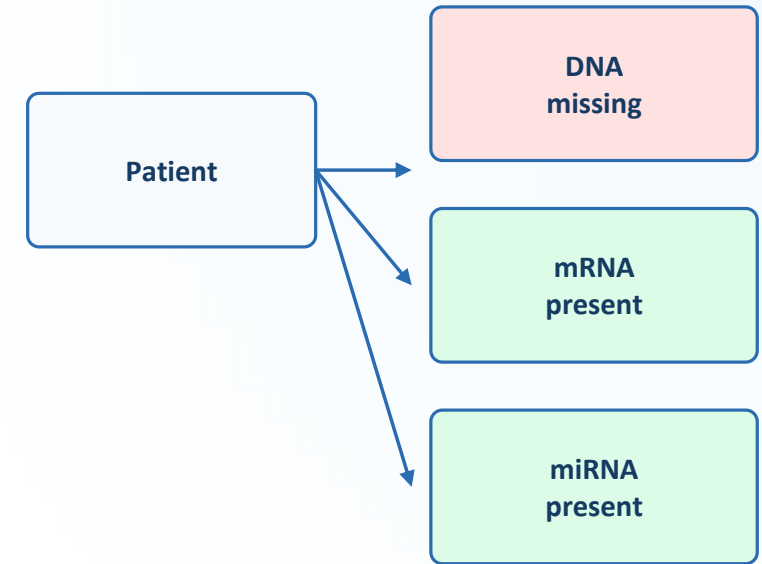
What is Multiomics?

Multi-omics integrates biological data by combining multiple layers—like genomics (DNA), transcriptomics (miRNA), and proteomics (proteins)—rather than looking at one in isolation.



The Problem Formulation

What if a patient has miRNA and mRNA information but not the DNA?



Potential Solutions

The main question driving the architecture is: "**Can we reconstruct this data?**" Exploring avenues to mathematically recover lost biological views.



What if we don't?

If we remove patients with incomplete data, we lose massive amounts of critical information, drastically shrinking cohort sizes.



Imputation Risks

If we fill the gaps carelessly (e.g., zero-filling or mean imputation), we introduce unreliable information and bias into the network.

The Methodology Landscape

Paradigm / Category	Representative Frameworks	Key Methodology & Mechanism	Strengths & Limitations	Ref
Traditional Graph Frameworks	Survey on GNNs	Mapped out foundational spatial and spectral splits.	Struggled with non-Euclidean complexities.	[1][2]
Late-Stage Fusion Networks	MOGONET	Avoided messy feature-space noise by utilizing View Correlation Discovery Networks (VCDN) to fuse independent omics label distributions.	Lacks natural handling for missing modal data blocks.	[3]
Latent Imputation & Borrowing	M ³ Care, MRGCN	Pioneered task-guided feature borrowing from graph neighbors and consensus indicator matrices.	Overcomes missing data without introducing raw-value noise.	[4][5]
Modality-Aware Attention	MAGNET	Deployed Patient-Modality Multi-Head Attention (PMMHA) with a binary mask to completely zero out missing streams.	Scales linearly with irregular missingness patterns.	[6]
Robust Joint Matrix Alternatives	MUSE (Unsupervised)	Uses ℓ_1 -norm regularized joint matrix factorization to mathematically isolate view-specific outliers while preserving local geometries.	Highly effective where GNNs fail under extreme noise.	[7]

Our Proposed Solution

Cross-Modal Learning

We learn the relationship from patients for whom both modalities are available, i.e. we do cross-modal learning.

How can we say this will work?

DNA and miRNA are different molecular levels of the genetic information of a patient. Because these molecular layers participate in interconnected regulatory mechanisms, information in one modality may be statistically informative about another.

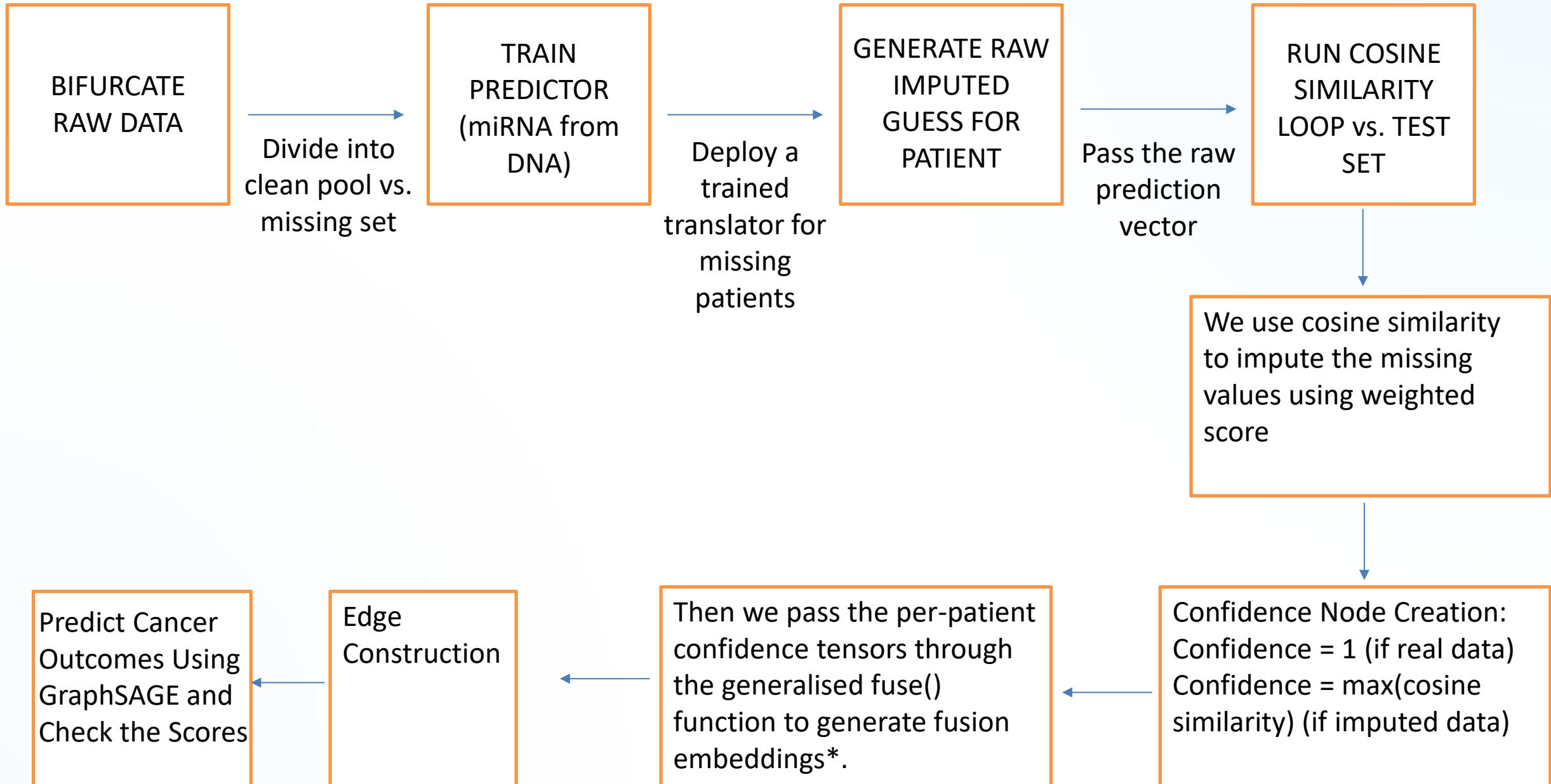
Verification & Reality Check

But how do we know the prediction is right?

We cannot directly verify a genuinely missing DNA profile.

Therefore, we must rely on a weighted confidence system that scales the model's trust based on the mathematically derived similarity between patients, ensuring predictions are heavily gated by empirical reliability.

Proposed Architecture Flow



Proposed Architecture

- **Edge Construction:** Edges are formed between patients based on available data similarities.
- **Confidence Node Creation:**
 - Confidence = 1 (if real data)
 - Confidence = max(cosine similarity) (if imputed data)
- We use cosine similarity to impute the missing values using a weighted score.
- Then we pass the per-patient confidence tensors through the generalised fuse() function to generate fusion embeddings.

● Step 1: Bifurcate

BIFURCATE RAW DATA

Divide the patient cohort into a clean pool vs. missing set based on modality availability.

● Step 2: Train

TRAIN PREDICTOR

(e.g., miRNA from DNA). Deploy a trained translator for patients with missing modalities.

● Step 3: Impute

GENERATE RAW IMPUTED
GUESS

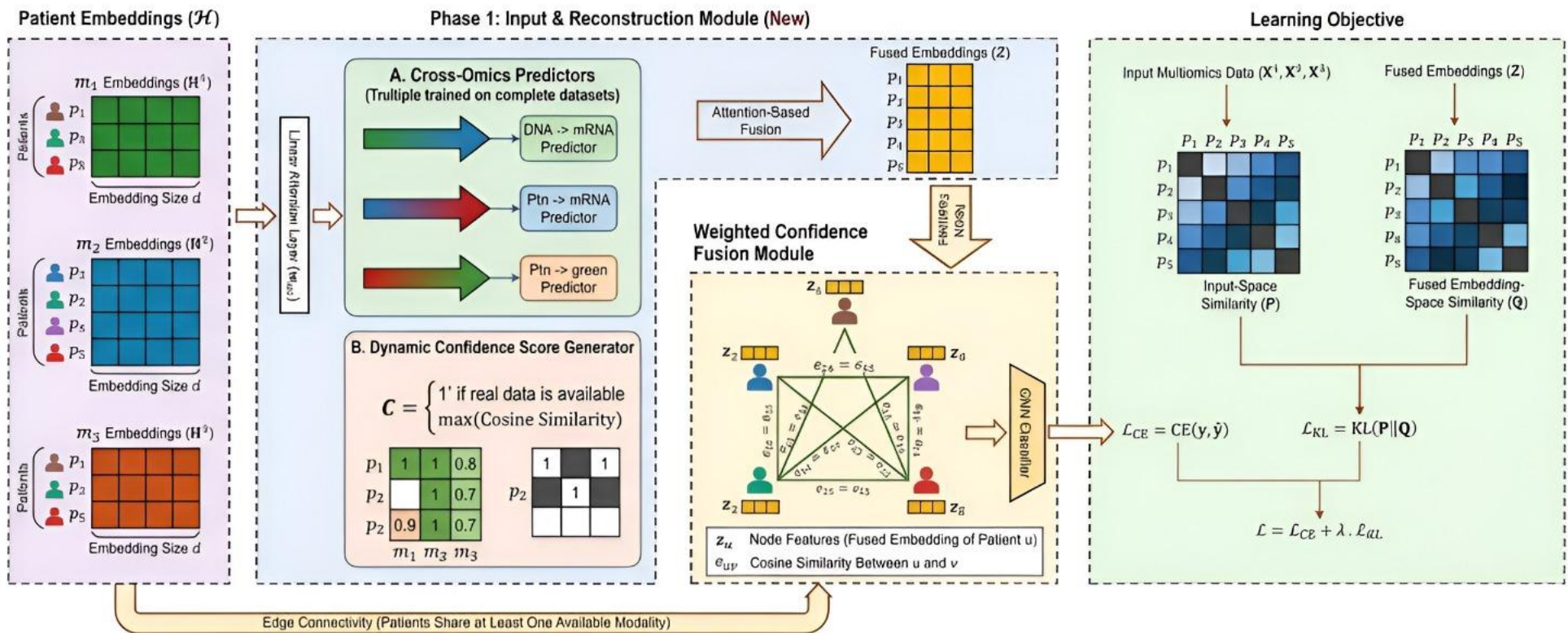
Pass the raw prediction vector for the patient to create a synthetic complete profile.

● Step 4: Validate

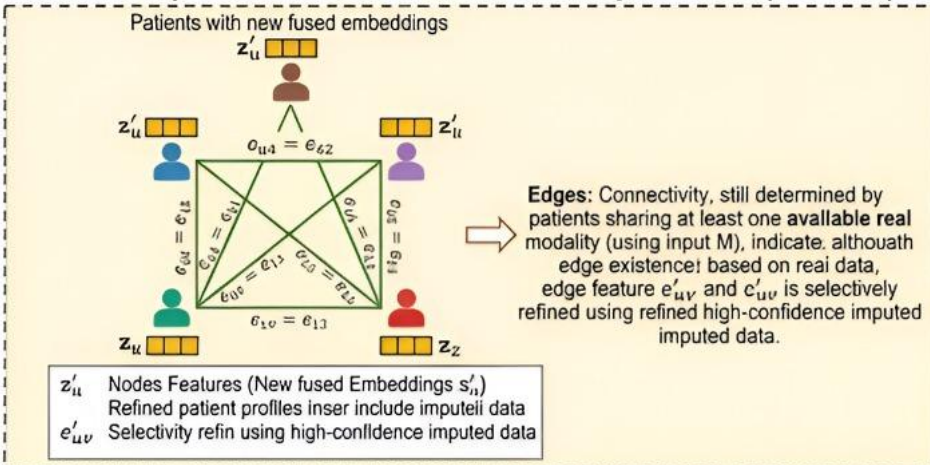
RUN COSINE SIMILARITY LOOP

Check against the test set and predict cancer outcomes using GraphSAGE while verifying scores.

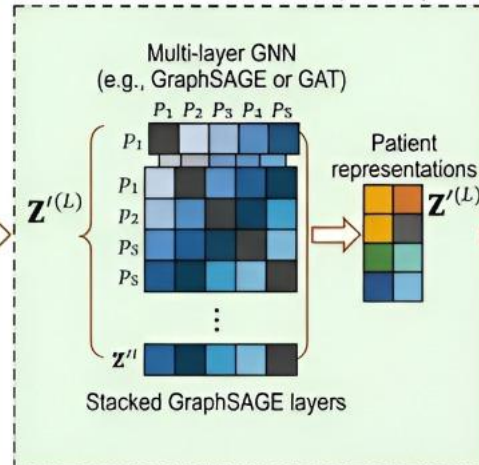
Architecture Diagram



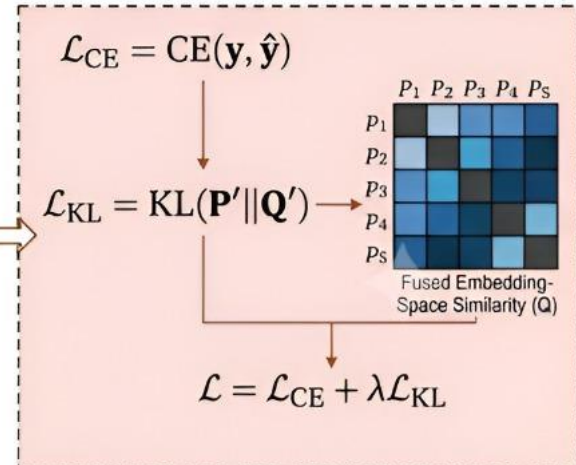
Phase 3: Updated Patient Interaction Graph Phase (Refined)



Phase 4: GNN Phase (Same)



Phase 5: Loss Function Phase (Refined)



Dataset Statistics

Dataset	#Patients	#Patients in Omics (Missing Rate %)			#Selected Features
		DNA	mRNA	miRNA	DNA
BRCA	956	328 (65.69%)	956 (0.00%)	584 (38.91%)	2,000
BLCA	433	431 (0.46%)	423 (2.31%)	426 (1.62%)	2,000
OV	360	356 (1.11%)	184 (48.89%)	302 (16.11%)	2,000

i Missing Rate (%) = Percentage of patients missing that modality.
DNA features are selected to a fixed size of 2,000 across all datasets for consistency.

(MiRNAToDNAPredictor)Pseudo Code

```
DEFINE NEURAL_NETWORK MiRNAToDNAPredictor(x_miRNA):
```

```
    // INPUT: miRNA feature vector
    // Layer 1
    h1 = LINEAR(x_miRNA, out_features=512)
    h1 = BATCH_NORMIALIZATION(h1)
    h1 = RELU(h1)
    h1 = DROPOUT(h1)

    // Layer 2
    h2 = LINEAR(h1, out_features=256)
    h2 = BATCH_NORMIALIZATION(h2)
    h2 = RELU(h2)
    h2 = DROPOUT(h2)

    // Output Layer
    output = LINEAR(h2, out_features=num_DNA_features)

    // Return predicted DNA vector
    RETURN output
```

```
DEFINE FUNCTION evaluate_predictions(y_true, y_pred):
```

```
    rmse = ROOT_MEAN_SQUARED_ERROR(y_true, y_pred)
    r2 = R2_SCORE(y_true, y_pred)

    cos_sims = EMPTY_LIST()
    FOR i FROM 1 TO NUMBER_OF_SAMPLES(y_true):
        cos_sim = COSINE_SIMILARITY(y_true[i], y_pred[i])
        APPEND(cos_sims, cos_sim)

    mean_cosine = MEAN(cos_sims)
    RETURN rmse, r2, mean_cosine, cos_sims
```

```
// CREATE COMPLETE DNA DATASET
```

```
real_DNA = AVAILABLE_DNA_SAMPLES
real_DNA["source"] = "real"
imputed_DNA["source"] = "imputed"
complete_DNA = CONCATENATE(real_DNA, imputed_DNA)
SAVE_TO_CSV(complete_DNA, dataset, split)
```

```
DEFINE FUNCTION train_model(model, train_loader, val_loader):
```

```
    criterion = MEAN_SQUARED_ERROR()
    optimizer = ADAMW(model.parameters(), lr=0.001)

    max_epochs = 100
    patience = 15
    best_val_loss = INFINITY
    patience_counter = 0
    best_model_state = NULL

    FOR epoch FROM 1 TO max_epochs:
        // ---- Training ----
        model.SET_TRAIN_MODE()
        total_train_loss = 0
        FOR (x_batch, y_batch) IN train_loader:
            x_batch, y_batch = MOVE_TO_DEVICE(x_batch, y_batch)
            y_pred = model.FORWARD(x_batch)
            loss = criterion(y_pred, y_batch)
            optimizer.ZERO_GRAD()
            loss.BACKWARD()
            optimizer.STEP()
            total_train_loss += loss.ITEM()

        avg_train_loss = total_train_loss /
            LENGTH(train_loader)

        // ---- Validation ----
        model.SET_EVAL_MODE()
        total_val_loss = 0
        WITH NO_GRADIENT():
            FOR (x_val, y_val) IN val_loader:
                x_val, y_val = MOVE_TO_DEVICE(x_val, y_val)
                y_val_pred = model.FORWARD(x_val)
                val_loss = criterion(y_val_pred, y_val)
                total_val_loss += val_loss.ITEM()
            avg_val_loss = total_val_loss / LENGTH(val_loader)

        // ---- Early Stopping ----
        IF avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            best_model_state = model.GET_STATE_DICT()
            patience_counter = 0
        ELSE:
            patience_counter += 1
            IF patience_counter >= patience:
                BREAK

        // Restore best model
        model.LOAD_STATE_DICT(best_model_state)
    RETURN model
```

```
// TEST THE MODEL
```

```
y_pred_test = model.PREDICT(test_miRNA)
y_pred_test = scaler_DNA.INVERSE_TRANSFORM(y_pred_test)
y_true_test = scaler_DNA.INVERSE_TRANSFORM(test_DNA)

rmse, r2, mean_cosine, cos_sims = evaluate_predictions(
    y_true_test, y_pred_test)

SAVE_METRICS_TO_CSV(rmse, r2, mean_cosine, cos_sims,
    dataset, split)
```

```
// CALCULATE CONFIDENCE SCORES
```

```
FOR each sample IN real_DNA:
    sample.cosine_similarity = 1.0
    sample.confidence_score = 1.0

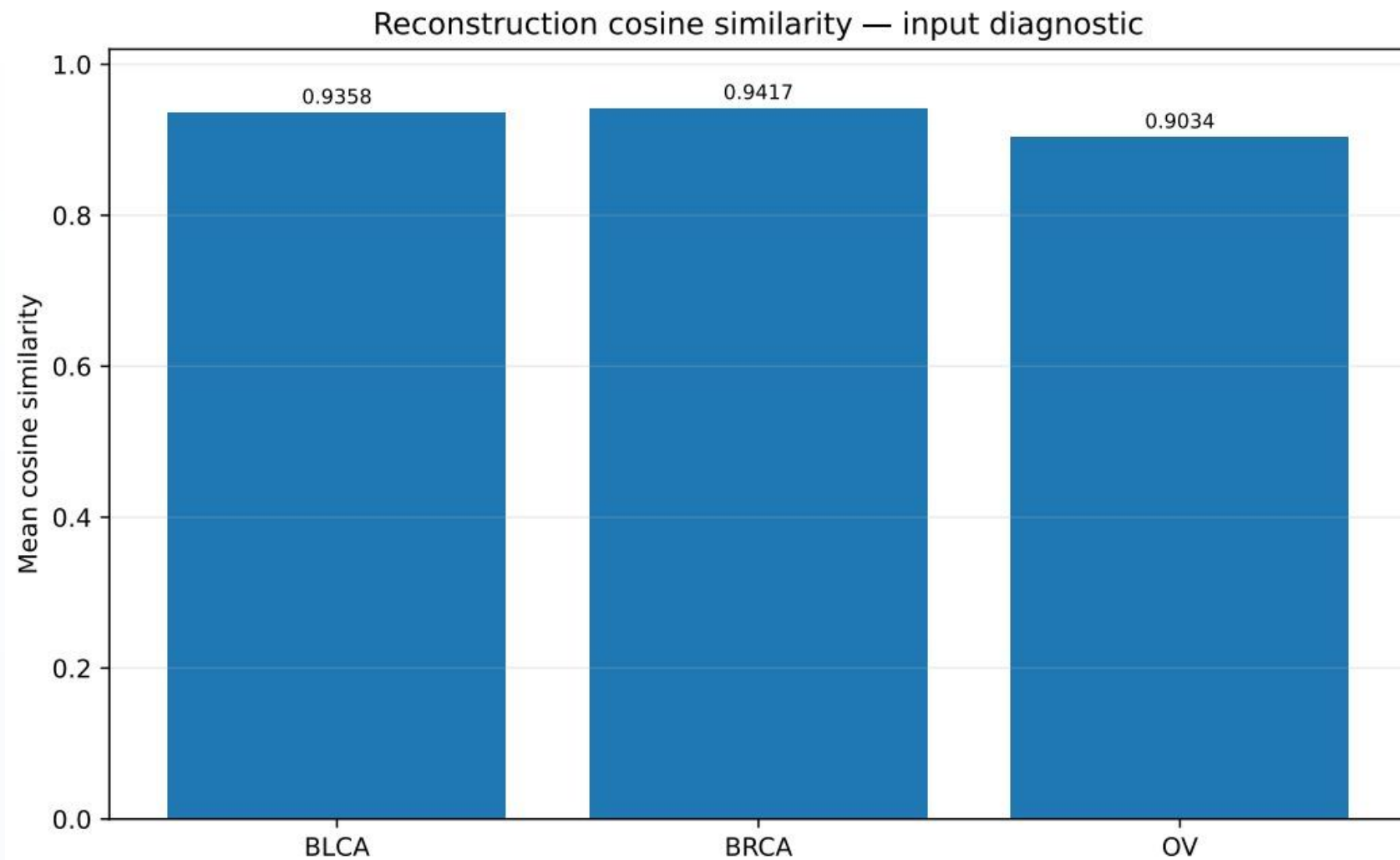
FOR each sample IN imputed_DNA:
    sample.cosine_similarity = mean_cosine
    sample.confidence_score = MAX(0, mean_cosine)

SAVE_CONFIDENCE_SCORES_TO_CSV(real_DNA, imputed_DNA, dataset, split)
```

```
// IMPUTE MISSING DNA
```

```
IF missing_miRNA IS NOT EMPTY:
    imputed_DNA = model.PREDICT(missing_miRNA)
    imputed_DNA = scaler_DNA.INVERSE_TRANSFORM(imputed_DNA)
    STORE_AS_IMPUTED(imputed_DNA)
ELSE:
    imputed_DNA = EMPTY_DATASET()
```

Results



fuse() Pseudo Code

```
FUNCTION fuse(x_proj, mask, mode="original", att_lin=None, shared_weights=None, external_confidence=None):

    // 1. Extract dimensions
    batch_size, num_heads, num_modalities, head_dims = SHAPE(x_proj)

    IF mode == "external":
        // NEW FRAMEWORK LOGIC: Inject per-patient confidence tensor
        REQUIRE external_confidence IS NOT NULL
        // Expand the external 2D tensor across all heads
        ext_conf = EXPAND(external_confidence, to_match_heads=num_heads)

        // Enforce mask so genuinely missing patients are respected
        ext_conf = ext_conf * mask

        // Normalize the confidence scores
        sum_conf = SUM(ext_conf, dimension=-1) CLAMPED TO MIN 1e-9
        att_weights = ext_conf / sum_conf

    ELSE:
        // Original, equal, and shared logic blocks
        ...

    // 3. Final masking and fusion execution
    att_weights = att_weights * mask
    fused_embeddings = SUM(att_weights * x_proj, dimension=modality_axis)

    RETURN fused_embeddings, att_weights
```

Stress Tests

Stress Test A (False Low Confidence):

For 20% of the patients, artificially drop the confidence tensor to ~ 0.1 on a modality that is genuinely present and valid. This evaluates the model's ability to gracefully degrade rather than crash.

Stress Test B (Noise + Falsely High Confidence):

For 20% of the patients, replace one modality's true embedding with Gaussian noise but force the model to trust it by keeping confidence high (~ 0.9). This tests vulnerability to spoofed confidence and corrupted data.

Empirical Data: BRCA

How Much Do We Trust The Model? Our trust in the framework stems from empirical data comparing it to the original model.

Framework	Condition	Accuracy	Macro-F1
Original	Unperturbed Control	0.8990 ± 0.0205	0.8806 ± 0.0200
Original	Stress Test A (False Low Conf)	0.8917 ± 0.0172	0.8725 ± 0.0165
Original	Stress Test B (Noise + High Conf)	0.8688 ± 0.0185	0.8440 ± 0.0191
New (Cosine Imp.)	Unperturbed Control	0.9010 ± 0.0147	0.8803 ± 0.0177
New (Cosine Imp.)	Stress Test A (False Low Conf)	0.8948 ± 0.0129	0.8743 ± 0.0169
New (Cosine Imp.)	Stress Test B (Noise + High Conf)	0.8708 ± 0.0185	0.8460 ± 0.0147

Empirical Data: BLCA

Framework	Condition	Accuracy	Macro-F1
Original	Unperturbed Control	0.9682 ± 0.0133	0.7941 ± 0.0915
Original	Stress Test A (False Low Conf)	0.9682 ± 0.0133	0.7941 ± 0.0915
Original	Stress Test B (Noise + High Conf)	0.9591 ± 0.0245	0.7084 ± 0.1882
New (Cosine Imp.)	Unperturbed Control	0.9705 ± 0.0116	0.8185 ± 0.0629
New (Cosine Imp.)	Stress Test A (False Low Conf)	0.9705 ± 0.0116	0.8185 ± 0.0629
New (Cosine Imp.)	Stress Test B (Noise + High Conf)	0.9682 ± 0.0085	0.7988 ± 0.0359

Based On Empirical Data...

- From table 1, we can conclude that the baseline performance is nearly identical between frameworks, but the **new framework shows slightly tighter variance** (0.8803 ± 0.0177 vs 0.8806 ± 0.0200 Macro-F1).
- Both frameworks exhibit remarkable resilience to artificially suppressed confidence scores (**Stress Test A**).
- The vulnerability to noise paired with falsely high confidence remains similar between both frameworks, with Macro-F1 dropping to ≈ 0.84 (**Stress Test B**).

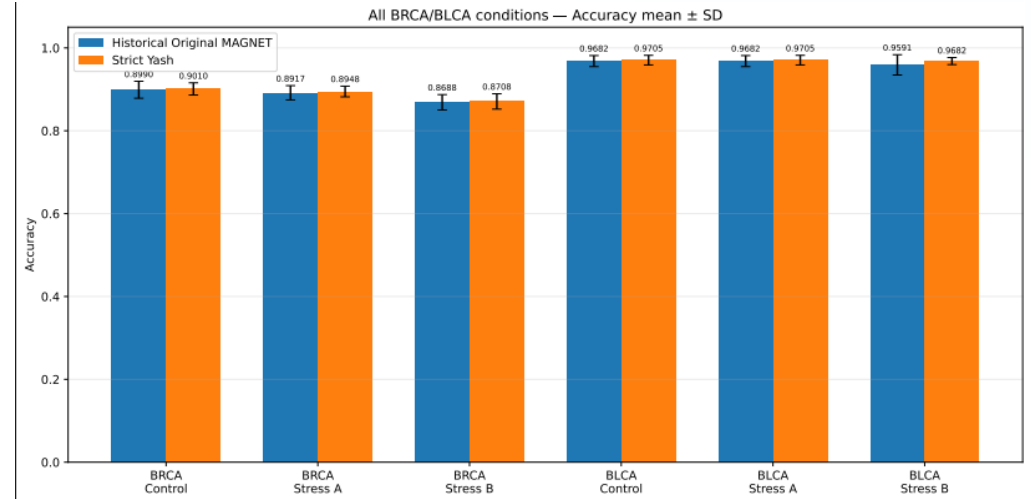
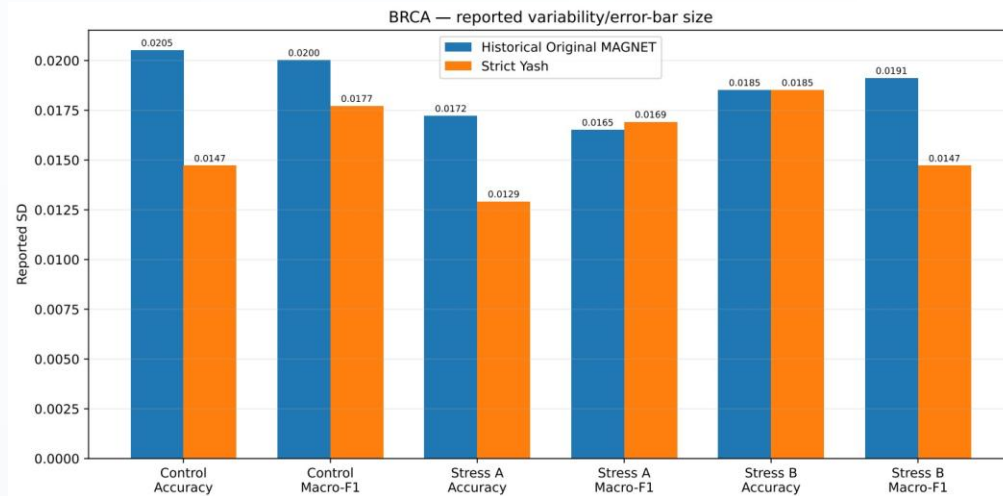
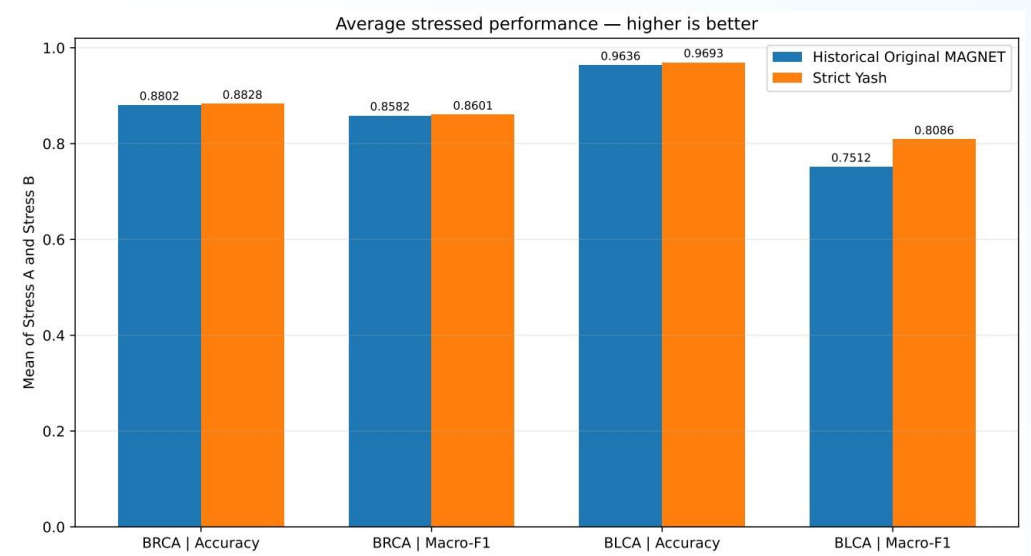
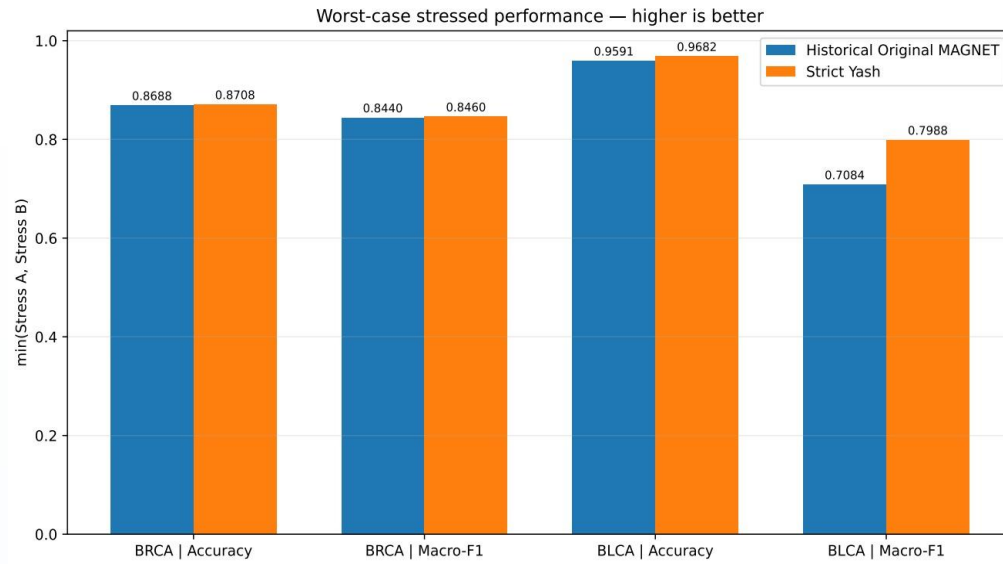
Empirical Data Analysis Continued...

- From table 2, we can conclude that the new framework provides a **notable boost to the baseline Macro-F1** (improving from 0.7941 to 0.8185) while **substantially reducing the variance** (± 0.0629 compared to the previous ± 0.0915). This suggests the imputed features provide a more cohesive training signal than simple masking.
- Both frameworks completely resisted the perturbation, showing no change in performance (**Stress Test A**).
- The original framework crashed spectacularly under Test B, with Macro-F1 plummeting to 0.7084 and variance exploding to ± 0.1882 . However, **the new framework demonstrates extraordinary resilience here**. Under the exact same noise injection, the Macro-F1 only dropped to 0.7988, and variance remained tightly controlled at ± 0.0359 (**Stress Test B**).

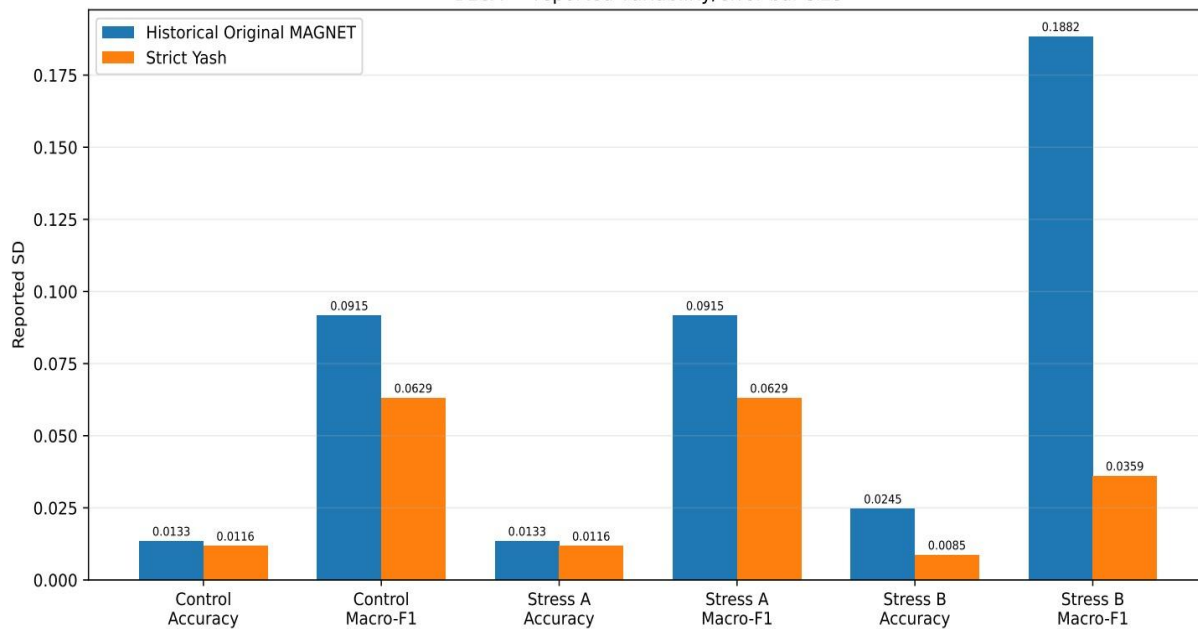
Conclusion of the Stress Tests

“ Cosine similarity imputation strategy, coupled with per-patient confidence tensor, acts as a powerful regularizer, particularly for highly sensitive datasets like BLCA. It prevents the model from blindly over-trusting corrupted embeddings, effectively neutralizing the catastrophic failure mode observed in the original architecture's stress tests.

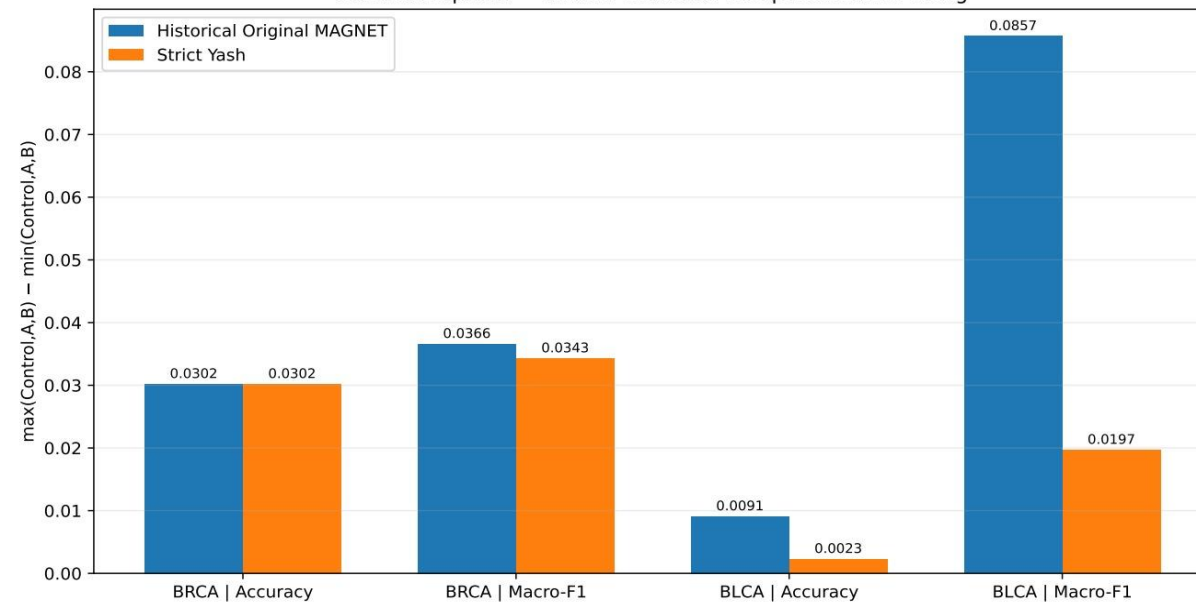
Results



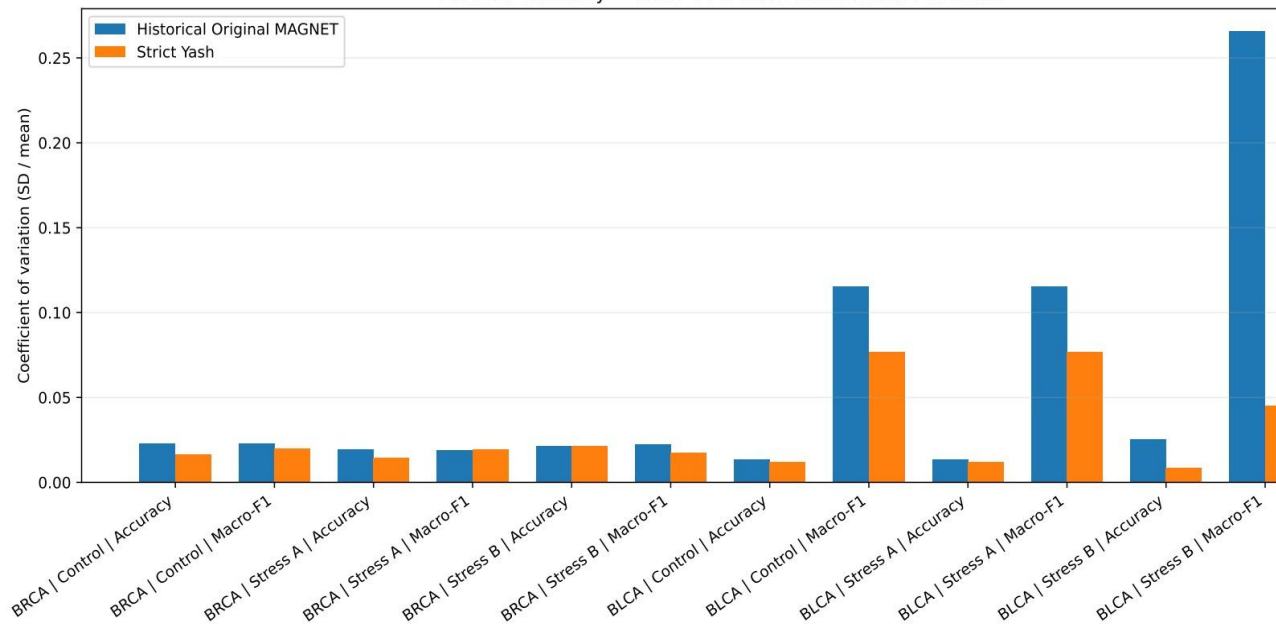
BLCA — reported variability/error-bar size



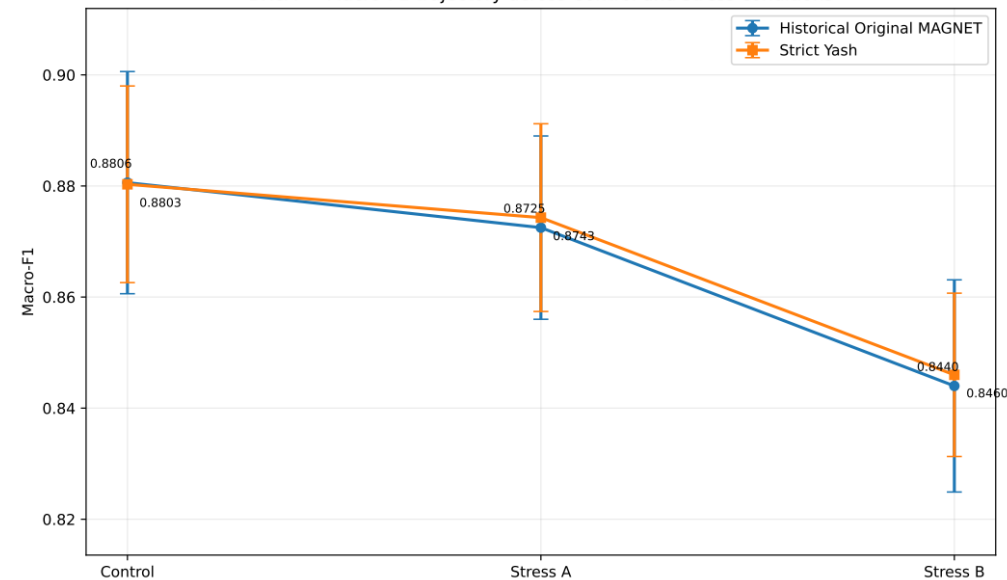
Condition spread — smaller indicates less performance swing

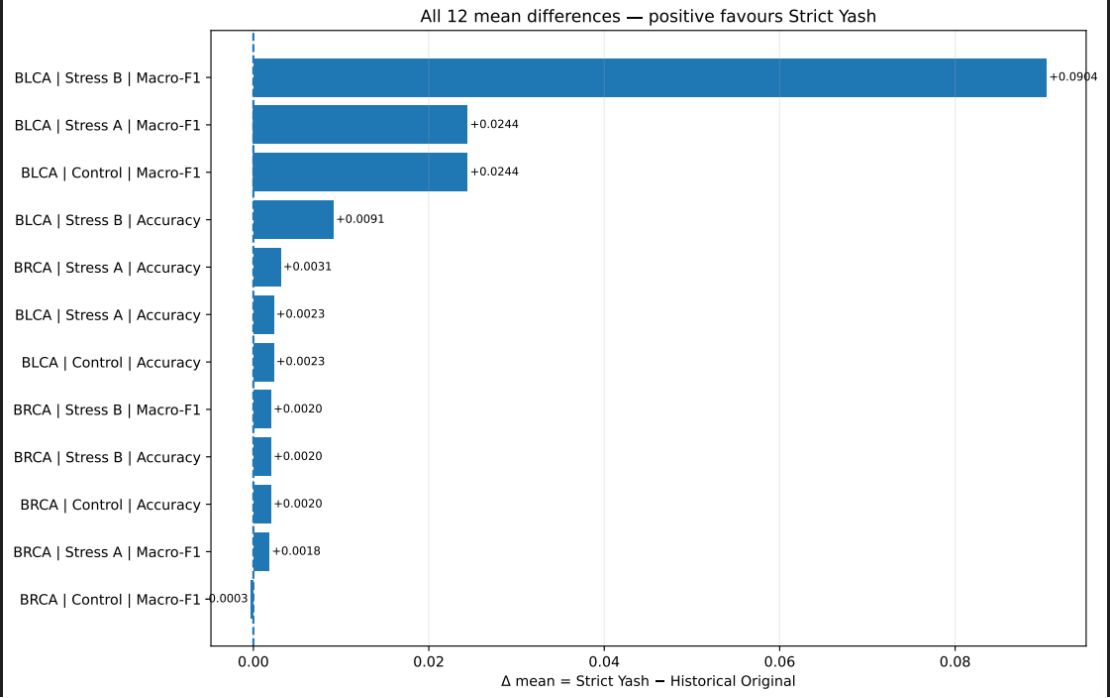
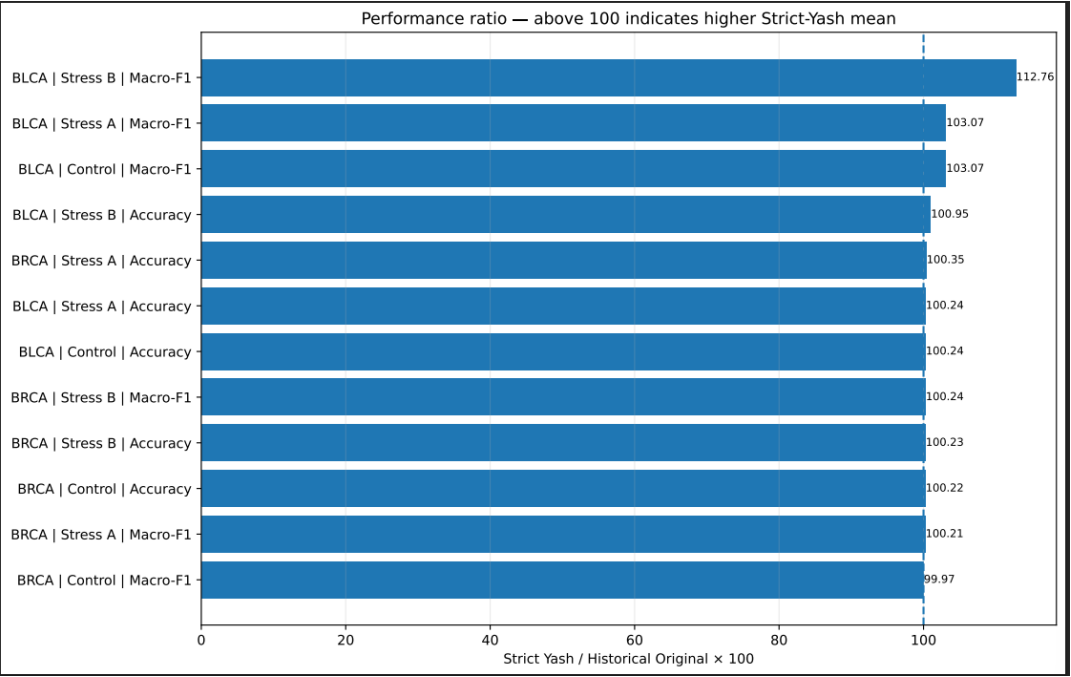


Relative variability — lower CV is more stable relative to mean



BRCA — Macro-F1 trajectory across Control and stress conditions





Summary & Next Steps

Core Conclusions

- Cross-omics features are highly directional and linear, except during multi-target upstream miRNA interactions.
- Multi-source late fusion in uncompressed feature spaces acts as an algorithmic noise filter, bypassing the need for heavy graph-based generative imputation model calculations.

Future Extensions

- Expand the graph edge construction methodology.
- Selectively incorporate high-confidence imputed modalities into patient-similarity edge weighting.
- Move beyond relying solely on strictly real modalities for graph formation.

References

- [1] Survey on GNNs: Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2020). A comprehensive survey on graph neural networks. IEEE Trans. Neural Networks and Learning Systems.
- [2] Multi-Omics Review: Reel, P. S., Reel, S., Pearson, E., Trucco, E., & Jefferson, E. (2021). Using machine learning approaches for multi-omics data analysis: A review. Biotechnology Advances.
- [3] MOGONET: Wang, T., et al. (2021). MOGONET integrates multi-omics data using graph convolutional networks... Nature Communications.
- [4] M3CARE: Zhang, C., et al. (2022). M3Care: Learning with Missing Modalities in Multimodal Healthcare Data. Proc. of 28th ACM SIGKDD.
- [5] MRGCN: Yang, B., Yang, Y., Wang, M., & Su, X. (2023). MRGCN: cancer subtyping with multi-reconstruction graph convolutional network... Bioinformatics.
- [6] MAGNET: Han, C. (2025). MAGNET: Multi-view graph autoencoder with cell-gene attention... PLOS Computational Biology.
- [7] MUSE: Bao, F., et al. (2022). Integrative spatial analysis of cell morphologies and transcriptional states with MUSE. Nature Biotechnology.
- [8] Experimental Reports & MAGNET Ablation Study (Tabakhi, S., et al., 2026). Implementation details on GitHub & Colab workspace.